Code exemple en langage JAVA

Référence : **SEL-GI-002** / Version : **1.0.0**/ Date : **06/03/2025**
Sécurité : **DIFFUSION RESTREINTE**

GIE SESAM-Vitale
5, boulevard Marie et Alexandre Oyon – 72019 LE MANS Cedex 2
www.sesam-vitale.fr

Conformément à l'article L.122-4 du Code de la Propriété Intellectuelle, toute représentation ou reproduction (intégrale ou partielle) du présent ouvrage, quel que soit le support utilisé, doit être soumise à l'accord préalable écrit de son auteur.

Il en est de même pour sa traduction, sa transformation, son adaptation ou son arrangement, quel que soit le procédé utilisé.

Tout manquement à ces obligations constituerait un délit de contrefaçon, au sens des articles L 335-2 et suivants du code de la propriété intellectuelle, susceptible d'entraîner des sanctions pour l'auteur du délit.

Historique des versions

Date	Version	Évolution du document
06/03/2025	V 1.0.0	Version initiale du document

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

Tables des matières

1 Introduction 5

1.1 Pré-requis 5
1.2 Documents et annexes externes 5
1.3 Contenu du document 5
1.4 Avertissement 5

2 Contenu du code exemple 6

2.1 Description des dossiers 6
2.2 Description des packages du code exemple 6
2.2.1 Package fr.sesamvitale.lps 6
2.2.2 Package fr.sesamvitale.pkcs12 7
2.2.3 Package fr.sesamvitale.psc 7
2.2.4 Package fr.sesamvitale.util 7
2.3 Description détaillée des classes du code exemple 7
2.4 Dossier src/main/resources 8

3 Librairie ClientTLSi-Assertion 8

3.1 Description des dossiers 8
3.2 Description des packages du code exemple 8
3.2.1 Package fr.santesocial.assertion 8
3.2.2 Package fr.santesocial.assertion.commun 8
3.2.3 Package fr.santesocial.assertion.cps 8
3.2.4 Package fr.santesocial.assertion.psc 8
3.2.5 Package fr.santesocial.assertion.structure 8
3.2.6 Package fr.santesocial.assertion.util 8
3.2.7 Package fr.santesocial.assertion.vitale 9
3.3 Description détaillée des classes du code exemple 9
3.4 Fichiers à la racine du projet 10

4 Authentifications 10

5 Webservices 10

6 Compilation du projet à partir des sources 10

6.1 Pré-requis 10
6.2 Génération des classes Java 10
6.3 Compilation du projet avec Eclipse 11
6.3.1 Installation de Eclipse 11
6.3.2 Librairies dépendantes 11
6.3.2.1 Projet ClientTLSi-Assertion 11
6.3.2.2 Projet tls_icir 11
6.3.3 Import du projet sous Eclipse 12
6.3.4 Compilation et exécution sous Eclipse 12
6.4 Exécution avec la carte CPS (authentification directe) 12
6.5 Exécution avec la carte Vitale 12

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

1 Introduction

1.1 Pré-requis

La lecture des spécifications d'interfaces du Service ainsi que du Cadre d'interopérabilité AMO est un pré-requis à la compréhension du code exemple.

1.2 Documents et annexes externes

Le tableau ci-dessous récapitule les principaux documents applicables. Dans l'ensemble du document, ils sont désignés par le code apparaissant dans la colonne « Référence ».

Référence	Document
CI	CI-TLSi_AMO_V1.2.2
ANX-WSDL	Archive zip SCHÉMAS ET WSDL ASSOCIÉS : CONTRAT_ICIR_EXP_1.5.0
SFG/GI	SEL-MP-043_04-00-02_INSi
MICA-MP-001	Manuel de programmation MICA 4.01.00

1.3 Contenu du document

Le document propose de décrire les deux composants proposés comme code exemple pour l'implémentation des appels aux webservices de recherche INSi ICIR :

- Le projet d'appel aux webservices en lui-même avec plusieurs modes d'authentification
- La librairie ClientTLSi-Assertion pour la mutualisation de la génération d'assertion conforme au cadre d'interopérabilité AMO

Le document fournit également les informations pour le paramétrage et l'exécution de ce code exemple.

1.4 Avertissement

Les codes sources sont fournis « tels quels », sans support.

La mise à disposition du code exemple ne saurait être interprétée comme un quelconque transfert des droits de propriété sur celui-ci.

L'utilisateur désigne ci-après l'entité destinataire du code exemple.

L'attention de l'utilisateur est appelée sur les modalités d'utilisation du code exemple.

Ce dernier est fourni à titre d'information permettant à l'utilisateur de réaliser librement l'adaptation personnalisée nécessaire à la création de l'interfaçage de son logiciel.

Le code exemple est transmis en son état de développement sans garantie, il n'a notamment pas fait l'objet de qualification sécuritaire.

Le code exemple ne fait l'objet d'aucune maintenance.

L'utilisateur est seul responsable des bonnes conditions de son utilisation et libre de s'inspirer des éléments fournis et de les adapter par ses propres moyens à la situation particulière de la solution logicielle qu'il développe.

Ainsi, il est déconseillé de procéder par voie de copier-coller du code à partir des exemples fournis.

L'éditeur DOIT adapter le code exemple fourni à son propre usage : les données fournies dans les flux exemples n'étant pas représentatives d'une utilisation en situation de production (certaines données sont renseignées « en dur » à titre d'exemple).

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

Il convient également de prendre en compte les contraintes techniques comme la présence d'un proxy sur le réseau.

2 Contenu du code exemple

Le code exemple propose l'implémentation d'un appel aux webservices INSi ICIR en se focalisant sur la génération des assertions SAML.

Le corps des requêtes est alimenté à partir de contenus statiques présents dans des fichiers *.xml.

2.1 Description des dossiers

ac_ans	Contient les fichiers utilisés pour vérifier le certificat du serveur lors de l'authentification mutuelle
Certificates	A pour objectif de contenir les certificats utilisés pour l'authentification mutuelle dans le cadre de Pro Santé Connect Doit être alimenter par l'utilisateur du code exemple
Certificates\INSI-AUTO	A pour objectif de contenir les certificats utilisés pour l'authentification mutuelle INSi pour «TLS-Mutuelle » Doit être alimenter par l'utilisateur du code exemple
Certificates\INSI-MANU	A pour objectif de contenir les certificats utilisés pour l'authentification mutuelle INSi pour «TLS-Mutuelle » Doit être alimenter par l'utilisateur du code exemple
Certificates\INSI-PSC	A pour objectif de contenir les certificats utilisés pour l'authentification mutuelle INSi pour « TODO TLS-Mutuelle » dans le cadre de Pro Santé Connect Doit être alimenter par l'utilisateur du code exemple
Config	Contient les fichiers de configuration. Actuellement, le fichier « pkcs11w64.p11 » contient la configuration de l'interface PKCS#11
src\main\java	Dossier contenant les sources des classes java d'exemples
src\main\resources	Dossier contenant les ressources utiles à l'exécution des classes d'exemples (fichiers de configurations, ...)
src\test\java	Dossier contenant les sources des classes java des tests JUnit des services (permet l'exécution des web-services)
Target\classes	Dossier de compilation des classes (créé lors de la première compilation du projet)
Target\generated-sources	Dossier contenant les sources des classes générées à partie des wsdl et de fichier de trace pour debug de flux
assertions	Dossier historiquement nécessaire au projet

2.2 Description des packages du code exemple

2.2.1 Package fr.sesamvitale.lps

Le package contient les principales classes du code exemple.

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

2.2.2 Package fr.sesamvitale.pkcs12

Le package contient les classes pour la gestion de certificats PKCS#12.

2.2.3 Package fr.sesamvitale.psc

Le package contient les classes pour la gestion de l'authentification Pro Santé Connect.

2.2.4 Package fr.sesamvitale.util

Le package contient des classes utilitaires.

2.3 Description détaillée des classes du code exemple

Nom du package	Nom de la classe java	Fonction
fr.sesamvitale.lps		Package principale pour le code exemple
	AssertionPSCHandler.java	Handler SOAP pour l'intégration d'une assertion PSC dans les requêtes
	AssertionPSHandler.java	Handler SOAP pour l'intégration d'une assertion CPS dans les requêtes
	AssertionVitaleHandler.java	Handler SOAP pour l'intégration d'une assertion Vitale dans les requêtes
	GetINS.java	Classe contenant la fonction principale pour l'exécution du code exemple
	MyTrustManager.java	Composant pour vérifier le certificat proposé par le serveur dans le cadre de l'authentification mutuelle
	AssertionException.java	Classe pour la gestion des exceptions levées à la génération d'assertions.
	AssertionPSBuilder.java	Alternative de génération d'une assertion CPS, n'est pas utilisé dans le code exemple
fr.sesamvitale.pkcs12		Package pour la gestion de l'authentification mutuelle avec un certificat *.p12
	Pkcs12.java	Interface pour la gestion de l'authentification mutuelle avec un certificat *.p12
	Pkcs12Impl.java	Implémentation par défaut de l'interface pour la gestion de l'authentification mutuelle avec un certificat *.p12
fr.sesamvitale.psc		Package dédié à la prise en compte des spécificités Pro Santé Connect
	ProxyJetonAM.java	Proxy pour obtenir le jeton AM nécessaire pour le mode d'authentification via Pro Santé Connect
	PscTrustManager.java	Trust-manager pour l'authentification mutuelle Pro Sant Connect
fr.sesamvitale.util		Package contenant des classes utilitaires
	AssertionSoapLogger.java	Handler SOAP pour la gestion des traces des requêtes et des réponses aux webservices. Fonctionne avec Log.java
	Log.java	Composant pour la gestion des traces des requêtes et des réponses aux webservices
	SSLKeyManagers.java	Gestionnaire de clé pour l'établissement du canal SSL
	TestCaractere.java	Classe utilitaire historiquement présente dans le code, nécessaire pour la manipulation de caractères dans le cadre de la génération d'assertions.

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

2.4 Dossier src/main/resources

Ce répertoire a pour rôle de recevoir le fichier *.wsdl et les fichiers *.xsd du webservice dans un sous-répertoire « wsdl » afin de permettre de générer les sources nécessaires à l'appel du webservice du code exemple.

3 Librairie ClientTLSi-Assertion

La libraire ClientTLSi-Assertion permet de mutualiser les classes de génération d'assertions pour faciliter leur usage dans les multiples TLSi AMO.

3.1 Description des dossiers

src\main\java	Dossier contenant les sources des classes java
src\main\resources	Dossier contenant les ressources utiles à l'exécution des classes (fichiers de configurations, ...)
src\test\java	Dossier contenant les sources des classes java des tests JUnit des services (permet l'exécution des web-services)
src\test\resources	Dossier contenant les ressources utiles à l'exécution des tests JUnit (fichiers de configurations, ...)
target	Dossier de compilation Maven

3.2 Description des packages du code exemple

3.2.1 Package fr.santesocial.assertion

La classe AssertionFactory est le point d'entrée dans la librairie pour la génération des assertions.

A noter que cette classe n'est pas utilisée dans le code exemple INSi ICIR.

3.2.2 Package fr.santesocial.assertion.commun

Le package contient le code partagé par l'ensemble des assertions.

3.2.3 Package fr.santesocial.assertion.cps

Le package contient les classes nécessaires à la génération d'assertion CPS

3.2.4 Package fr.santesocial.assertion.psc

Le package contient les classes nécessaires à la génération d'assertion CPS dans le cadre de Pro Santé Connect

3.2.5 Package fr.santesocial.assertion.structure

Le package contient les classes nécessaires à la génération d'assertion Structure.

3.2.6 Package fr.santesocial.assertion.util

Le package contient des classes utilitaires.

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

3.2.7 Package fr.santesocial.assertion.vitale

Le package contient les classes nécessaires à la génération d'assertion Vitale.

3.3 Description détaillée des classes du code exemple

Nom du package	Nom de la classe java	Fonction
fr.santesocial.assertion		Package contenant la fabrique à assertion
	AssertionFactory	Classe utilisée comme point d'entrée pour la génération d'assertion. A noter que cette classe n'est pas utilisée dans le code exemple INSi ICIR
fr.santesocial.assertion.commun		Package contenant les composants de base pour la génération d'une assertion SAML
	AbstractAssertionBuilder.java	Classe de base de création d'une assertion SAML
	AbstractAssertionSigneeBuilder.java	Représentation d'un composant pour la création d'assertion signée
	AssertionAttribute.java	Classe de gestion d'un attribut SAML dans une assertion
	AssertionAttributeStatement.java	Classe de gestion de l'élément SAML AttributeStatement contenant les attributs d'une assertion
	AssertionException.java	Exception dans la création d'une Assertion SAML
	AssertionSignature.java	Génération de l'objet XML-Signature d'un document
	AssertionSignedInfo.java	Classe de gestion de l'élément SignedInfo de l'objet XML-Signature de l'assertion
	ISignatureFactory.java	Interface de la fabrique de signature
fr.santesocial.assertion.cps		Package pour la génération d'assertion CPS
	Cps3Pkcs11.java	Classe singleton qui encapsule les appels à la cryptolib CPS
	CpsAssertionBuilder.java	Construction d'une Assertion CPS au standard SAML
	CpsInfo.java	Informations d'une CPS
	CpsSignatureFactory.java	Fabrique de signature CPS
	DialogSaisieCodePorteur.java	Classe pour une saisie via une boîte de dialogue du code porteur de la carte CPS. A noter que cette classe n'est pas utilisée dans le code exemple INSi ICIR
fr.santesocial.assertion.psc		Package pour la génération d'assertion Pro Santé Connect
	PscAssertionBuilder.java	Construction d'une Assertion Pro Santé Connect au standard SAML
fr.santesocial.assertion.structure		Package pour la génération d'assertion Structure
	StructureAssertionBuilder.java	Construction d'une Assertion Structure au standard SAML
fr.santesocial.assertion.util		Package contenant des classes utilitaires
	ChiffrementUtil.java	Classe utilitaire concernant le chiffrement
	Conversion.java	Classe permettant de convertir différentes données

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

	TestCaractere.java	Classe permettant de tester un type de caractère
fr.santesocial.assertion.vitale		Package pour la génération d'assertion Vitale
	VitaleAssertionBuilder.java	Construction d'une assertion Vitale au standard SAML
	VitaleInfo.java	Informations d'une carte Vitale
	VitaleSignatureFactory.java	Fabrique de signature Vitale

3.4 Fichiers à la racine du projet

pom.xml	Déclaration du projet pour Maven
structure.ini	Fichier exemple pour la configuration nécessaire à la génération d'une assertion Structure

4 Authentifications

Il existe trois modes d'authentification, l'authentification avec la carte CPS (authentification directe), l'authentification avec un certificats établissement (authentification indirecte) et l'authentification PSC (authentification directe avec Pro Santé Connect).

Il convient d'adapter la configuration en fonction du mode d'authentification :

- Le choix de l'URL du webservice
- Les certificats logiciels à exploiter
- Le chemin vers la cryptolib CPS installée sur le poste

5 Webservices

Le code exemple propose l'implémentation des fonctions de recherche INSi (ICIR):

- rechercherInsAvecTraitsIdentite
- rechercherInsAvecCarteVitale

6 Compilation du projet à partir des sources

6.1 Pré-requis

Pour compiler le projet, vous devez disposer de maven 3.6 et de Java JDK version 17.

Décompresser l'archive du projet vers un dossier nommé à votre convenance (ici « insi_icir »).

6.2 Génération des classes Java

Les classes représentant les web services ont été générées grâce au plugin jaxws-maven-plugin en compilant l'application via « mvn clean compile » dans le dossier target\generated-sources.

Les WSDL sont fournis dans une annexe externe [ANX-WSDL].

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

6.3 Compilation du projet avec Eclipse

6.3.1 Installation de Eclipse

Voir le site http://www.eclipse.org/

Remarque :

Le projet a été testé avec Eclipse « 2023-09 (4.29.0) », mais il est probable qu'il fonctionne aussi sur des versions antérieures (sans garantie).

6.3.2 Librairies dépendantes

6.3.2.1 Projet ClientTLSi-Assertion

Les librairies dépendantes sont définies à travers le pom.xml du projet :

librairie	Version
fr.santesocial.mica:jnamica	2.01.01
org.bouncycastle:bcpkix-jdk18on org.bouncycastle:bcprov-jdk18on	1.76
org.opensaml:opensaml-core org.opensaml:opensaml-saml-api org.opensaml:opensaml-saml-impl	4.0.1
iaik.pkcs.pkcs11.wrapper:iaikPkcs11Wrapper	1.4
net.java.dev.jna:jna	5.7.0
org.apache.logging.log4j:log4j-core	2.21.1
org.slf4j:slf4j-reload4j	2.0.9
junit:junit	4.13.2

6.3.2.2 Projet tls_icir

Les librairies dépendantes sont définies à travers le pom.xml du projet :

librairie	Version
fr.santesocial.lps:ClientTLSi-Assertion	4.0.0
jakarta.validation:jakarta.validation-api	3.0.2
jakarta.xml.bind:jakarta.xml.bind-api	4.0.0
com.sun.xml.bind:jaxb-impl	4.0.0
jakarta.xml.ws:jakarta.xml.ws-api	4.0.0
com.sun.xml.ws:jaxws-rt	4.0.0
com.fasterxml.jackson.core:jackson-databind com.fasterxml.jackson.core:jackson-annotations	2.14.2
org.apache.logging.log4j:log4j-core	2.21.1

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale

6.3.3 Import du projet sous Eclipse

Pour importer le projet sous Eclipse :

1. Pour importer un Maven projet déjà existant : **File→Import** … ;
2. **Maven / Existing Maven Project**;
3. Sélectionner le dossier « insi_icir» décompressé, puis **Finish**

6.3.4 Compilation et exécution sous Eclipse

Sous Eclipse :

- Le projet se compile via le JDK 17;
- Les « Main » d'exemple peuvent être lancés directement sous Eclipse (carte CPS : voir le chapitre suivant pour la limitation possible en fonction de la version de la cryptoLib).

6.4 Exécution avec la carte CPS (authentification directe)

Afin de pouvoir exécuter les exemples sous Eclipse, il est nécessaire d'utiliser idéalement la dernière version de la Cryptolib (5.2.2).

Sur les anciennes versions de CryptoLib (inférieure à la version 3.06), en raison d'un classpath trop long formé par Eclipse, l'exécution d'une classe utilisant la carte CPS n'arrivera pas à son terme, à cause d'une limitation du buffer de la dll cps_pkcs11_w32 qui ne supportait pas plus de 4096 caractères, et terminera l'exécution sans aucun code retour.

Note :

Dans le cas d'une exécution des tests avec la carte CPS, il est nécessaire de renseigner le code PIN correct de la carte CPS dans la constante CODE_PORTEUR de la classe GetINS.java. **Dans le cas contraire, il y un risque de blocage de la carte CPS par l'exécution des tests (le code PIN livré par défaut dans le fichier de configuration est « 1234 »)**.

Il peut également être nécessaire de modifier le numéro de slot attribué au lecteur PC/SC de la carte CPS (la constante CPS_SLOT de la classe GetINS.java est renseigné à « 4 »).

6.5 Exécution avec la carte Vitale

Afin de pouvoir exécuter les exemples sous Eclipse, il est nécessaire d'utiliser la version 4.01.00 de MICA en 64 bits.

Cette version doit être installée dans C:\Program Files\santesocial\mica\.

Ce document à diffusion restreinte ne peut être diffusé sans l'autorisation d'une personne habilitée au GIE SESAM-Vitale